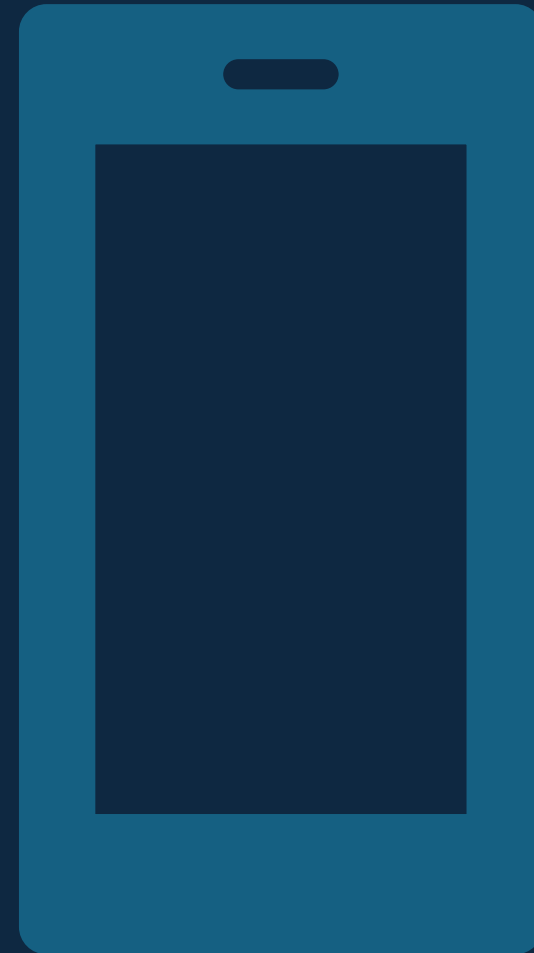


Fundamentos e Interface Mobile

Material
MOBILE_006



DESENVOLVIMENTO
MOBILE NATIVO VS.
HÍBRIDO VS. CROSS
PLATFORM:

- **Nativo:** Código específico para cada plataforma (Swift/Objective-C para iOS, Kotlin/Java para Android). Performance máxima, mas custo e tempo de desenvolvimento maiores.
- **Híbrido (Web View):** Usa tecnologias web (HTML, CSS, JS) dentro de um container nativo (WebView). Mais rápido de desenvolver, mas performance pode ser limitada. Ex: Cordova, Ionic antigo.
- **Cross-Platform (Compilado/Ponte):** Escreve-se código em uma linguagem (ex: JavaScript com React Native, Dart com Flutter) que é compilado/interpretado para gerar interfaces e lógica nativas. Busca o "melhor dos dois mundos": desenvolvimento mais rápido que nativo puro, performance muito superior a híbrido.

TIPOS DE APLICAÇÕES MOBILE

Compare as abordagens e escolha a melhor estratégia para o seu projeto

 NATIVO Aplicações para uma única plataforma	 HÍBRIDO Aplicações com uma única base de código	 CROSS-PLATFORM Aplicações para múltiplas plataformas
 PRINCIPAIS PRÓS ✓ Alto desempenho ✓ Experiência de usuário nativa	 PRINCIPAIS PRÓS ✓ Desenvolvimento mais rápido ✓ Base de código compartilhada	 PRINCIPAIS PRÓS ✓ Código reutilizável ✓ Custo reduzido
 CONTRAS ✗ Maior custo de desenvolvimento ✗ Código específico da plataforma	 CONTRAS ✗ Desempenho geralmente inferior ✗ Dependência de plugins nativos	 CONTRAS ✗ Desempenho menor que nativo ✗ Recursos específicos limitados
 TECNOLOGIAS TÍPICAS Swift, Kotlin	 TECNOLOGIAS TÍPICAS Ionic, Cordova	 TECNOLOGIAS TÍPICAS React Native, Flutter



QUAL ESCOLHER?



NATIVO

Ideal para apps que exigem o máximo de performance e integração com o dispositivo.



HÍBRIDO

Ideal para projetos que precisam de agilidade e compartilham recursos entre plataformas.



CROSS-PLATFORM

Ideal para alcançar várias plataformas com economia e código reutilizável.

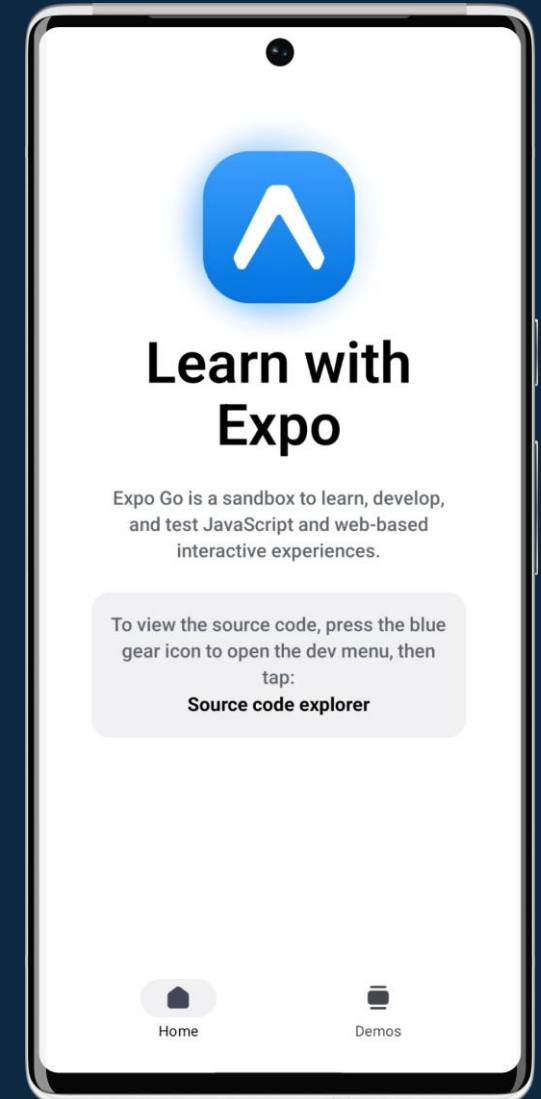
React Native

- Framework criado pelo Facebook.
- Permite construir apps mobile usando React (JavaScript/JSX) e conceitos de desenvolvimento web.
- "Learn once, write anywhere": A lógica pode ser compartilhada, mas a interface pode precisar de ajustes.
- Renderiza componentes de UI nativos, não WebViews (geralmente). Isso resulta em melhor performance e look-and-feel nativo.
- Usa uma "ponte" (Bridge) para comunicar entre o JavaScript e o código nativo.

EXPO

O que é o Expo ?

- **Um conjunto de ferramentas e serviços** construídos em cima do React Native.
- **Objetivo:** Simplificar e acelerar o desenvolvimento.
- **Expo Go App:** Permite visualizar e testar seu app no dispositivo físico sem precisar de Android Studio ou Xcode configurados (inicialmente).
- **Expo CLI:** Ferramenta de linha de comando para criar, iniciar e gerenciar projetos Expo.
- **Expo SDK:** Biblioteca de APIs pré-construídas que dão acesso fácil a funcionalidades do dispositivo (câmera, localização, notificações, etc.) sem precisar mexer com código nativo diretamente



Expo

Fluxo de Trabalho

- Fluxos de Trabalho (Workflows):
 - **Managed Workflow (Gerenciado):** O mais simples. Você escreve apenas JavaScript/TypeScript. Expo cuida de toda a complexidade nativa. Ideal para iniciantes e muitos casos de uso. **É o que usaremos.**
 - **Bare Workflow (Básico):** Dá mais flexibilidade, permite incluir código nativo customizado, mas exige configuração de Android Studio/Xcode. É como um projeto React Native "puro" com algumas bibliotecas Expo opcionais.



Vantagens

- ▶ Vantagens de usar o Expo Vantagens de usar o Expo

Vantagens de usar o Expo

- Configuração rápida do ambiente.
- Desenvolvimento e testes acelerados (Expo Go).
- Acesso fácil a APIs nativas.
- Builds simplificados na nuvem (EAS - Expo Application Services).
- Atualizações Over-The-Air (OTA)



Introdução e Configuração do Ambiente (Windows)

- Para desenvolver com React Native usando Expo, não precisamos instalar o Android Studio pesado imediatamente, pois usaremos o **Expo Go** no celular físico para testar.

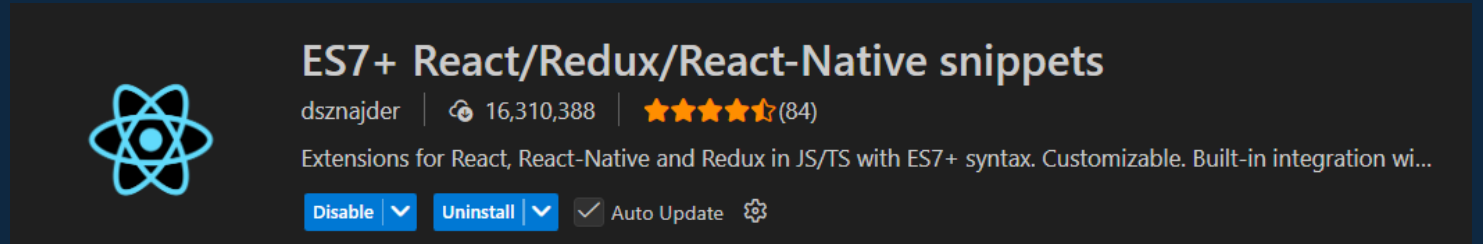


Passo 1: Instalação do Node.js (LTS)

- O React Native precisa do Node.js para rodar o servidor de desenvolvimento (Metro Bundler).
 1. Acesse **nodejs.org**.
 2. Baixe a versão LTS (Long Term Support) – é a mais estável.
 3. Instale no Windows (Next, Next, Finish).
 4. Abra o PowerShell ou CMD e digite **node -v** para confirmar.

Passo 2: Visual Studio Code (VS Code)

- Baixe e instale o VS Code.
- Extensão Recomendada: Instale a extensão **ES7+ React/Redux/React-Native snippets** para facilitar a escrita de código.



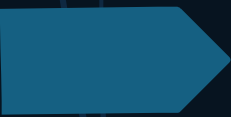


Passo 3: Expo CLI e Expo Go

- Antigamente, instalávamos o Expo globalmente.
- Hoje, a recomendação oficial é usar o **npx** (executor de pacotes do Node) para criar o projeto, garantindo sempre a versão mais atual.
- No Celular (Android ou iOS):
 - Vá na loja de aplicativos e instale o app **Expo Go**.
 - Garanta que o celular e o computador **estejam na mesma rede Wi-Fi**.

Criando o Primeiro Projeto

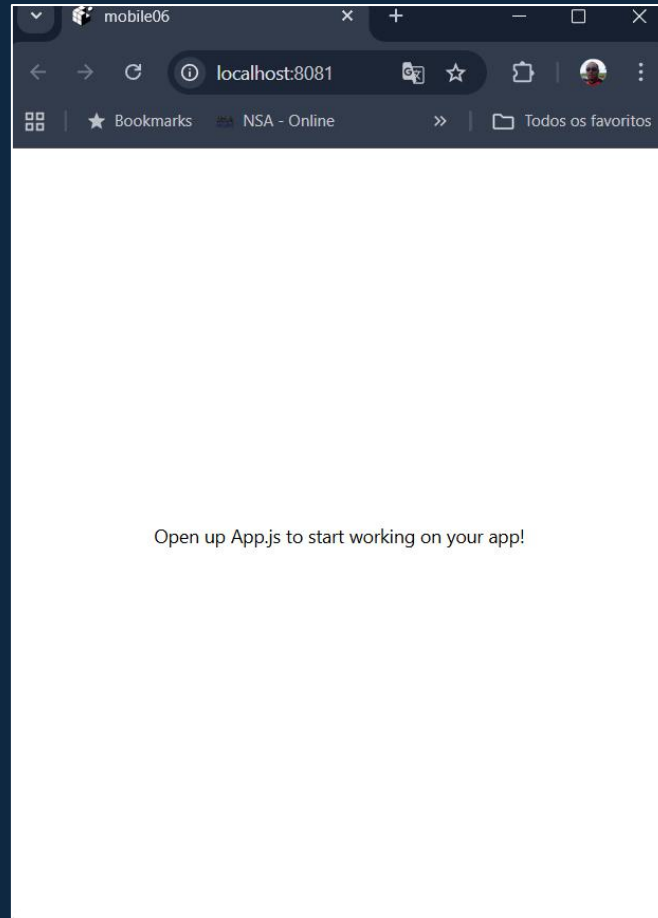
No terminal do VS Code ou
PowerShell:



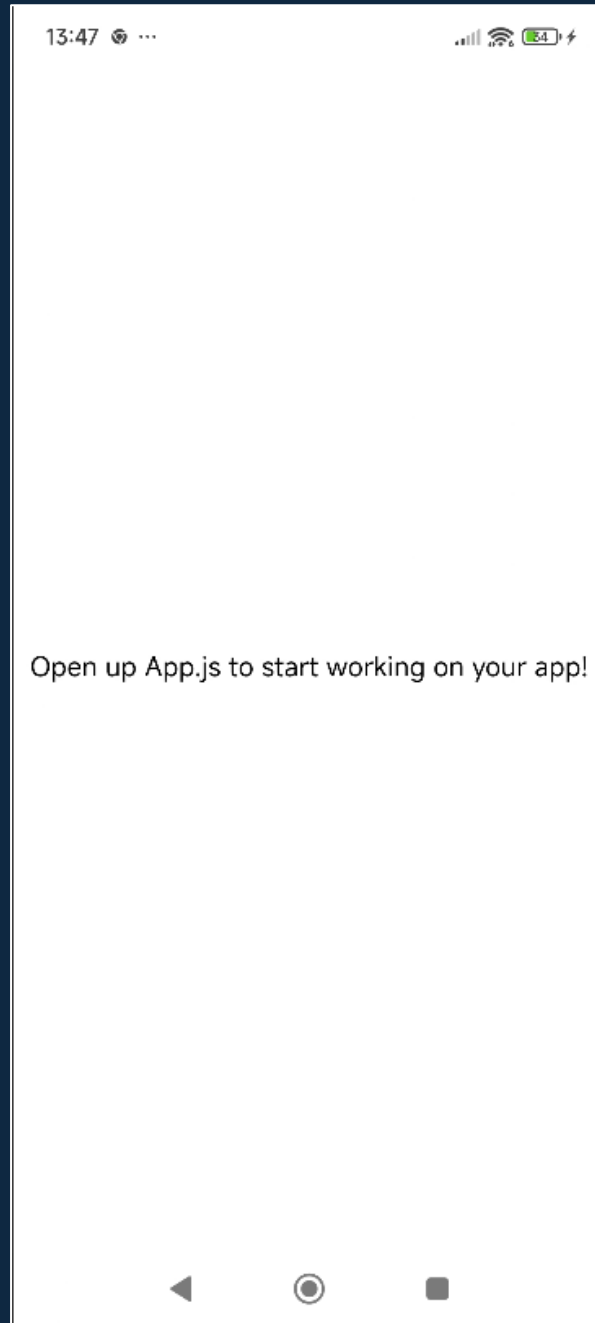
Criando o Primeiro Projeto

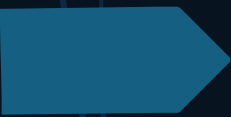
- No powershell ou cmd digite:
`npx create-expo-app mobile06 --template blank`
- Entra na pasta
`cd mobile06`
- Para testes em sala de aula, podemos usar html para rodar no chrome:
`npx expo install react-dom react-native-web @expo/metro-runtime`
- Inicia o servidor
`npx expo start`

Web



Mobile





Diferenças: Web vs. Mobile (JSX)

- No React Native, não usamos HTML. Usamos componentes nativos que o React "traduz" para a linguagem do celular (Java/Kotlin ou Swift/Obj-C).

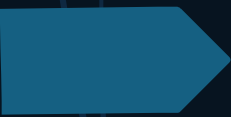
Tabela comparativa

HTML (Web)	React Native	Descrição
<code><div></code>	<code><View></code>	O container padrão. Não tem rolagem automática.
<code></code> , <code><p></code> , <code><h1></code>	<code><Text></code>	Todo texto deve estar dentro desta tag. Texto solto causa erro.
<code><button></code> , <code><a></code>	<code><Button></code> , <code><Pressable></code> , <code><TouchableOpacity></code>	Componentes tocáveis.
<code>onClick</code>	<code>onPress</code>	Evento de toque na tela.
<code></code>	<code><Image></code>	Requer a prop source.

Exemplo Prático (Comparação):

```
// Web (HTML/React.js)
<div className="container" onClick={handleClick}>
  <h1>Olá Mundo</h1>
</div>

// Mobile (React Native)
<Pressable style={styles.container} onPress={handlePress}>
  <Text style={styles.title}>Olá Mundo</Text>
</Pressable>
```



Estilização no Mobile: Flexbox e StyleSheet

- No React Native, não existe arquivo CSS separado (como .css).
- Usamos objetos JavaScript para estilizar. O sistema de layout padrão é o **Flexbox**.

Diferença Crucial do Flexbox



Web: O padrão do flexDirection é row (linha / horizontal).



React Native: O padrão do flexDirection é column (coluna / vertical). Isso ocorre porque a rolagem do celular é vertical.

Exemplo Completo: Layout e Flexbox

- Crie este código no seu App.js:

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <View style={styles.boxVermelho}>
        <Text style={styles.texto}>Box 1</Text>
      </View>
      <View style={styles.boxAzul}>
        <Text style={styles.texto}>Box 2</Text>
      </View>
    </View>
  );
}
```

Exemplo Completo: Layout e Flexbox

```
const styles = StyleSheet.create({
  container: {
    flex: 1, // Ocupa a tela toda
    backgroundColor: '#f0f0f0',
    alignItems: 'center', // Alinha horizontalmente (eixo
    // cruzado)
    justifyContent: 'center', // Alinha verticalmente
    // (eixo principal)
  },
  boxVermelho: {
    width: 100,
    height: 100,
    backgroundColor: 'red',
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 20,
  },
```

```
  boxAzul: {
    width: 100,
    height: 100,
    backgroundColor: 'blue',
    justifyContent: 'center',
    alignItems: 'center',
  },
  texto: {
    color: '#fff',
    fontWeight: 'bold',
  }
});
```

Resultado

13:57



Box 1

Box 2

Nota sobre Bibliotecas de Estilo

- Embora o StyleSheet seja o padrão (e o melhor para aprender performance), no mercado usa-se muito:
- **NativeWind:** Usa as classes do Tailwind CSS (ex: `className="flex-1 bg-red-500"`).
- **Styled Components:** Sintaxe CSS-in-JS (ex: `const Container = styled.View...`).

Inputs e Listas (Otimização)



TextInput (Formulários)

Diferente da web, o texto digitado não vai automaticamente para uma variável. Precisamos usar Estado (useState).



FlatList (Substituindo o Map e RecyclerView)

Nunca use `.map()` para listas longas no mobile (isso trava o celular pois renderiza itens fora da tela). Use **FlatList**, que renderiza apenas o que o usuário está vendo (**lazy loading**).

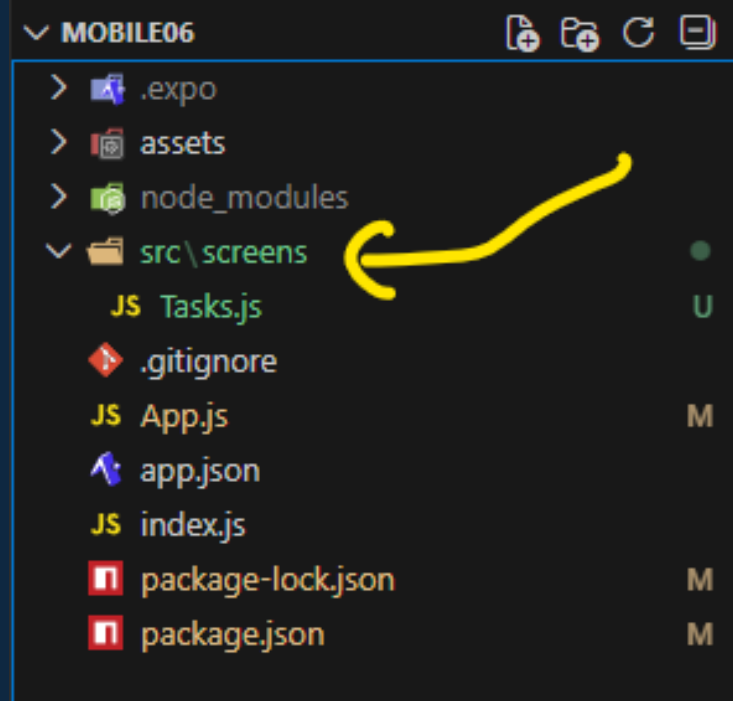


Prática comum

- Como o Expo é configurado para procurar sempre o arquivo App.js como ponto de partida (entry point), o "truque" é importar uma nova tela dentro do App.js e renderizá-la sozinha, ignorando temporariamente o resto.
- Vamos criar um componente chamado Tasks.js para ser nossa lista básica de tarefas para testarmos Inputs e Listas.

Crie o arquivo do Componente (Tasks.js)

- Primeiro, crie a pasta e o arquivo para organizar o projeto, mas sem apagar nada do que já fez.
 - Crie uma pasta: src/screens (boa prática).
 - Crie o arquivo: src/screens/Tasks.js.



Tasks.js

Nos próximos slides temos um:

Exemplo Completo:

- Lista de Tarefas (Input + FlatList)



```
import React, { useState } from 'react';
import {
  StyleSheet, Text, View, TextInput, Button, FlatList, StatusBar
} from 'react-native';

export default function Tasks() {
  const [tarefa, setTarefa] = useState('');
  const [lista, setLista] = useState([]);

  function adicionarTarefa() {
    if (tarefa === '') return;

    // Adiciona um objeto com id único (usando Date para exemplo simples)
    setLista([...lista, { id: Date.now().toString(), texto: tarefa }]);
    setTarefa(''); // Limpa o input
  }

  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />

      <Text style={styles.titulo}>Minhas Tarefas</Text>
    </View>
  );
}
```



```
<View style={styles.inputContainer}>
  <TextInput
    style={styles.input}
    placeholder="Digite uma tarefa..."
    onChangeText={(valor) => setTarefa(valor)} // Atualiza o estado
    value={tarefa}
  />
  <Button title="Adicionar" onPress={adicionarTarefa} />
</View>

<FlatList
  data={lista} // 1. O array de dados
  keyExtractor={(item) => item.id} // 2. A chave única de cada item
  renderItem={({ item }) => ( // 3. Como renderizar cada item
    <View style={styles.itemLista}>
      <Text>{item.texto}</Text>
    </View>
  )}
  ListEmptyComponent={<Text>Nenhuma tarefa cadastrada.</Text>}
/>
</View>
);
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 50, // Evita o notch do celular
    backgroundColor: '#fff',
  },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  inputContainer: {
    flexDirection: 'row',
    marginBottom: 20,
    gap: 10,
  },
  input: {
    flex: 1,
    borderWidth: 1,
    borderColor: '#ccc',
    padding: 10,
    borderRadius: 5,
  },
  itemLista: {
    padding: 15,
    backgroundColor: '#e0e0e0',
    marginBottom: 10,
    borderRadius: 5,
  }
});
```



Link para o
Tasks.js

https://drive.google.com/file/d/18wICrPcf1cQaQLkM2O_5mXzjsqbgW_kb/view?usp=sharing

Altere o App.js para teste:

```
import React from 'react';
import { StyleSheet, View, Text } from 'react-native';

// 1. IMPORTANTE: Importe o seu novo componente
import Tasks from './src/screens/Tasks';

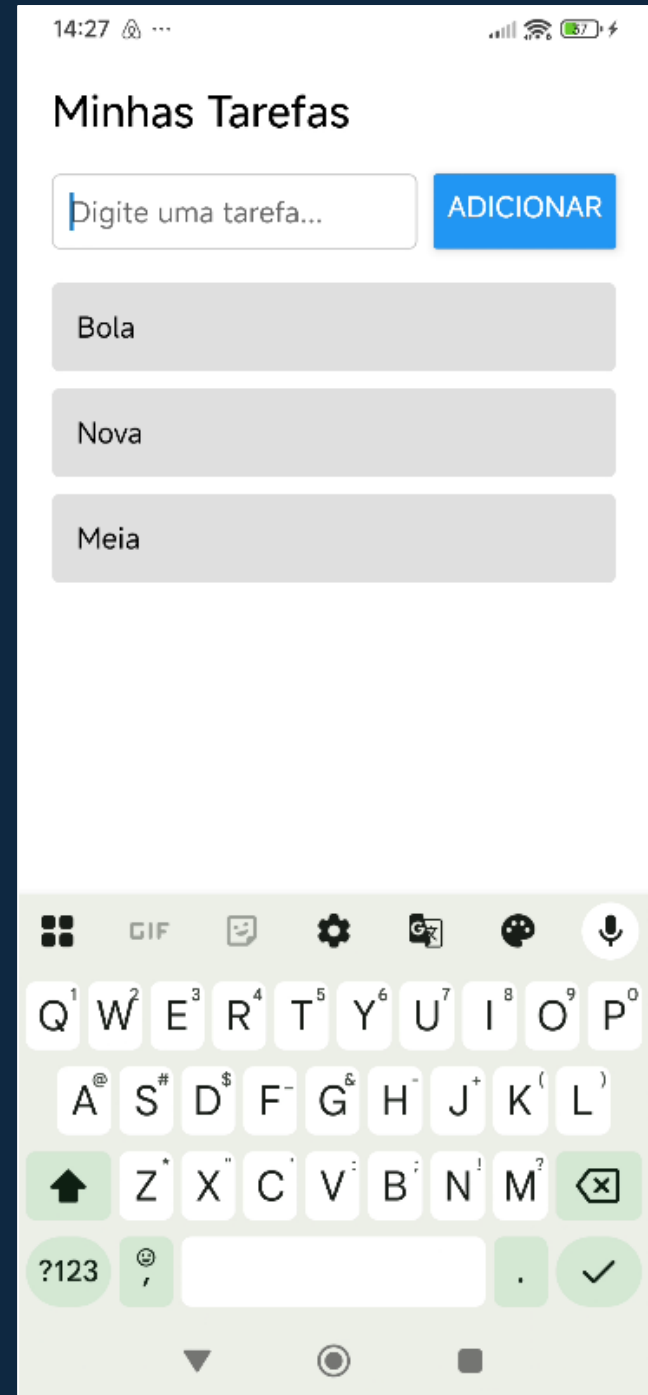
export default function App() {

  // --- MODO DE TESTE ---
  // Se quiser testar a tela Tasks, descomente a linha abaixo:
  return <Tasks />;

  // -----
  // O CÓDIGO ABAIXO SERÁ IGNORADO ENQUANTO O RETURN ACIMA EXISTIR
  // -----

  return (
    <View style={styles.container}>
```

Resultado





Resumo da Aula

- Instalamos Node e Expo CLI.
- Criamos o app e rodamos no celular via QR Code.
- Aprendemos que `<View>` substitui `<div>`.
- Usamos Flexbox (lembrando que é coluna por padrão).
- Implementamos uma lista performática com FlatList.

Fim



Prof. Maromo